Lecture 11 Support Vector Machine II

STAT 441: Statistical Methods for Learning and Data Mining

Wenlu TANG

Department of Mathematical and Statistical Sciences
University of Alberta

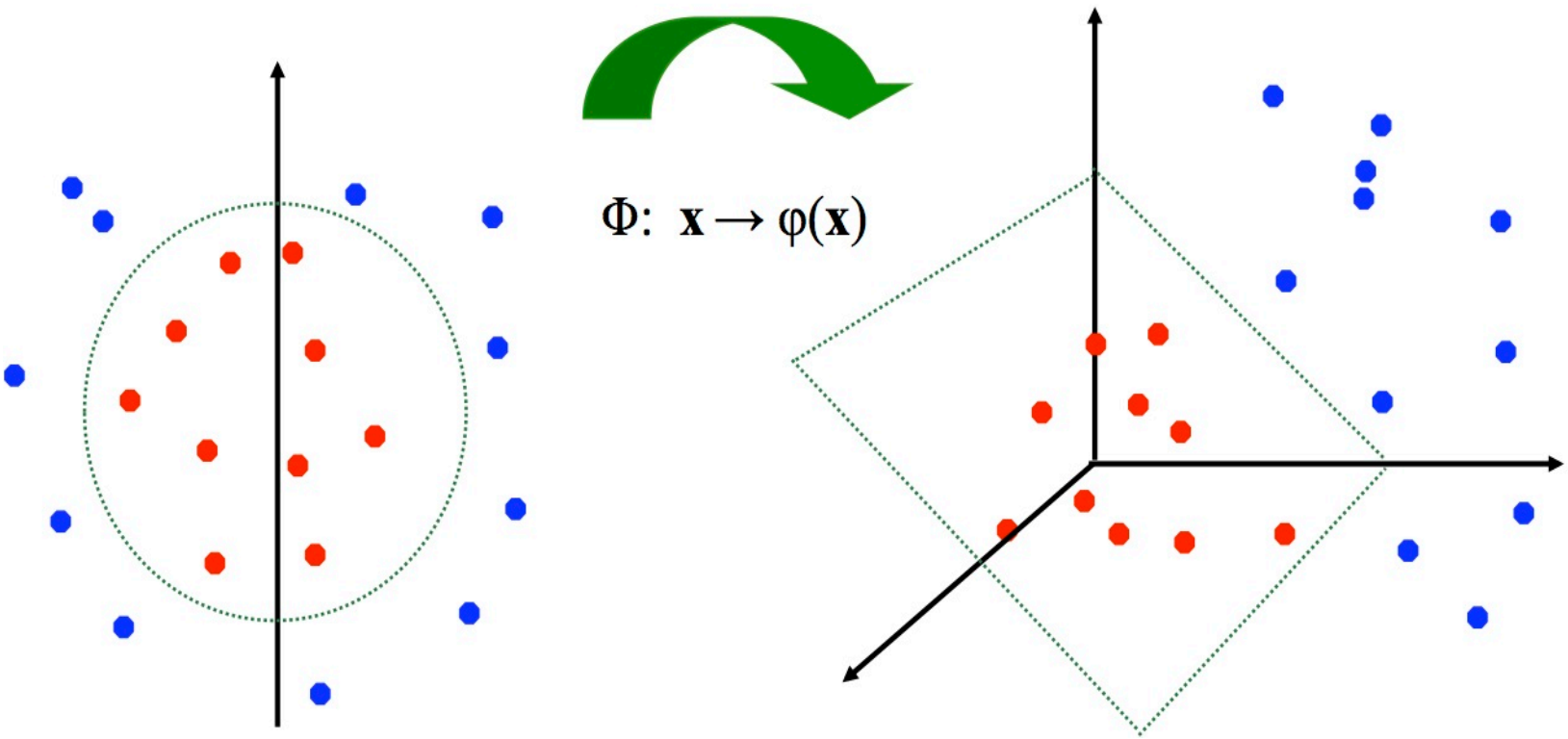
Winter, 2026

Outline

1. Feature Expansion
2. Kernel Trick
3. More than 2 classes
4. Summary and Remark

Non-linear SVMs: Feature Expansion

- General idea: the original feature space can always be mapped to some higher-dimensional feature space where the training set is separable



UNIVERSITY OF ALBERTA
EDMONTON·ALBERTA·CANADA

Feature Expansion

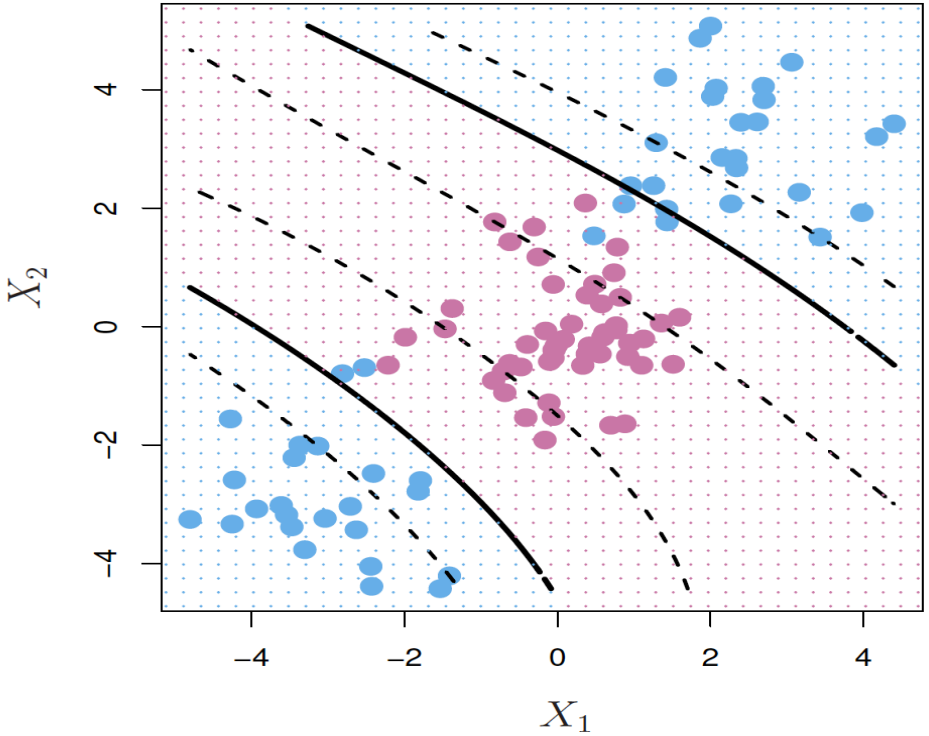
- Enlarge the space of features by including transformations; for example $X_1^2, X_2^3, X_1X_2, X_1X_2^2, \cdots$, Hence go from a p-dimensional space to a $M > p$ dimensional space.
- Fit a support-vector classifier in the enlarged space.
- This results in non-linear decision boundaries in the original space.
- Example: Suppose we use $(X_1, X_2, X_1^2, X_2^2, X_1X_2)$ instead of just (X_1, X_2). Then the decision boundary would be of the form

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 + \beta_5 X_1 X_2 = 0.$$

- This leads to nonlinear decision boundaries in the original space (quadratic conic sections).

Cubic Polynomials

- Here we use a basis expansion of cubic polynomials — from 2 variables to 9.
- The support-vectorclassifier in the enlarged space solves the problem in the lower-dimensional space



- The decision boundary is

$$\beta_0+\beta_1X_1+\beta_2X_2+\beta_3X_1^2+\beta_4X_2^2+\beta_5X_1X_2+\beta_6X_1^3+\beta_7X_2^3+\beta_8X_1X_2^2+\beta_9X_1^2X_2=0.$$

Nonlinearities and Kernels

- Polynomials (especially high-dimensional ones) get wild rather fast.
- There is a more elegant and controlled way to introduce nonlinearities in support vector classifier — through the use of kernels.
- Before we discuss these, we must understand the role of inner products in support vector classifier.

UNIVERSITY OF ALBERTA
EDMONTON·ALBERTA·CANADA

Inner products and kernels

- Inner product between vectors

$$\langle x_i, x_{i'} \rangle = \sum_j x_{ij} x_{i''j}.$$

- The linear support vector classifier can be represented as

$$f(x) = \beta_0 + \sum_i \alpha_i \langle x, x_i \rangle$$

- To estimate parameters $\alpha_1, \cdots, \alpha_n$ and β_0, all we need are $\binom{n}{2}$ inner products $\langle x, x_i \rangle$ between all pairs of training observations.
- It turns out that most of the $\hat{\alpha}_i$ can be zero

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \hat{\alpha}_i \langle x, x_i \rangle,$$

where $\mathcal{S}$ is the support set of indices i such that $\hat{\alpha}_i > 0$.

UNIVERSITY OF ALBERTA
EDMONTON·ALBERTA·CANADA

The"Kernel Trick"

- The linear classifier relies on inner product between vectors $K(x_i, x_j) = x_i^\top x_j$
- If every datapoint is mapped into high-dimensional space via some transformation $\Phi : x \to \phi(x)$, the inner product becomes:

$$K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$$

- A kernel function is a function that is eqiuvalent to an inner product in some feature space.

UNIVERSITY OF ALBERTA
EDMONTON·ALBERTA·CANADA

The"Kernel Trick"

Example:

- 2-dimensional vectors $\boldsymbol{x} = [x_1\ x_2]$; let $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = (1 + \boldsymbol{x}_i^\top \boldsymbol{x}_j)^2$
- Need to show that $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x}_j)$:

$$\begin{aligned}
K(\boldsymbol{x}_i, \boldsymbol{x}_j) =& \phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x}_j) \\
=& 1 + x_{i1}^2 x_{j1}^2 + 2x_{i1}x_{j1}x_{i2}x_{j2} + x_{i2}^2 x_{j2}^2 + 2x_{i1}x_{j1} + 2x_{i2}x_{j2} \\
=& [1\ x_{i1}^2\ \sqrt{2}x_{i1}x_{i2}\ x_{i2}^2\ \sqrt{2}x_{i1}\ \sqrt{2}x_{i2}]^\top [1\ x_{j1}^2\ \sqrt{2}x_{j1}x_{j2}\ x_{j2}^2\ \sqrt{2}x_{j1}\ \sqrt{2}x_{j2}] \\
=& \phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x}_j),
\end{aligned}$$

where $\phi(\boldsymbol{x}) = [1\ x_1^2\ \sqrt{2}x_1x_2\ x_2^2\ \sqrt{2}x_1\ \sqrt{2}x_2]$.

- Thus, a kernel function implicitly maps data to a high-dimensional space (without the need to compute each $\phi(\boldsymbol{x})$ explicitly).

UNIVERSITY OF ALBERTA
EDMONTON·ALBERTA·CANADA

What Functions are Kernels?

- For some functions $K(\boldsymbol{x}_i, \boldsymbol{x}_j)$ checking that $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x}_j)$ can be cumbersome.
- Mercer's theorem: Every semi-positive definite symmetric function is a kernel
- Example:
  - Linear: $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \boldsymbol{x}_i^\top \boldsymbol{x}_j$
    - Mapping Φ: $x \to \phi(x)$, where $\phi(x)$ is x itself
  - Polynomial of power p: $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = (1 + \boldsymbol{x}_i^\top \boldsymbol{x}_j)^p$
    - Mapping Φ: $x \to \phi(x)$, where $\phi(x)$ has $\binom{p+d}{p}$ dimensions.

Kernels and Support Vector Machine

- If we can compute inner products between observations, we can fit a support vector classifier — can be very abstract!
- Some special kernel function can do this for us. E.g.

$$K(x_i, x_{i'}) = (1 + \sum_j x_{ij} x_{i"j})^2$$

computes the inner products needed for d dimensional polynomials — $\binom{p+d}{d}$ basis functions!

- The solution has the form

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \hat{\alpha}_i K(x, x_i).$$

UNIVERSITY OF ALBERTA
EDMONTON·ALBERTA·CANADA

Radial Kernel

- x_{ij} is the j-th feature of the **first** point.
- $x_{i'j}$ is the j-th feature of the **second** point.
- The subtraction $(x_{ij} - x_{i'j})$ is finding the distance between those two specific points for that specific feature.

- The radial Kernel has the format

$$K(x_i, x_{i'}) = \exp\left(-\gamma \sum_j (x_{ij} - x_{i'j})^2\right),$$

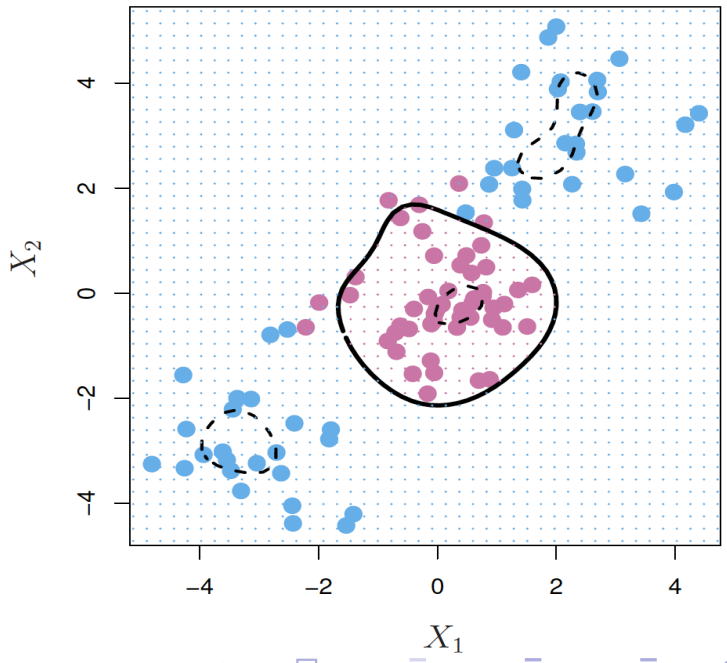
where γ is tuning parameter.

- The decision bounady is,

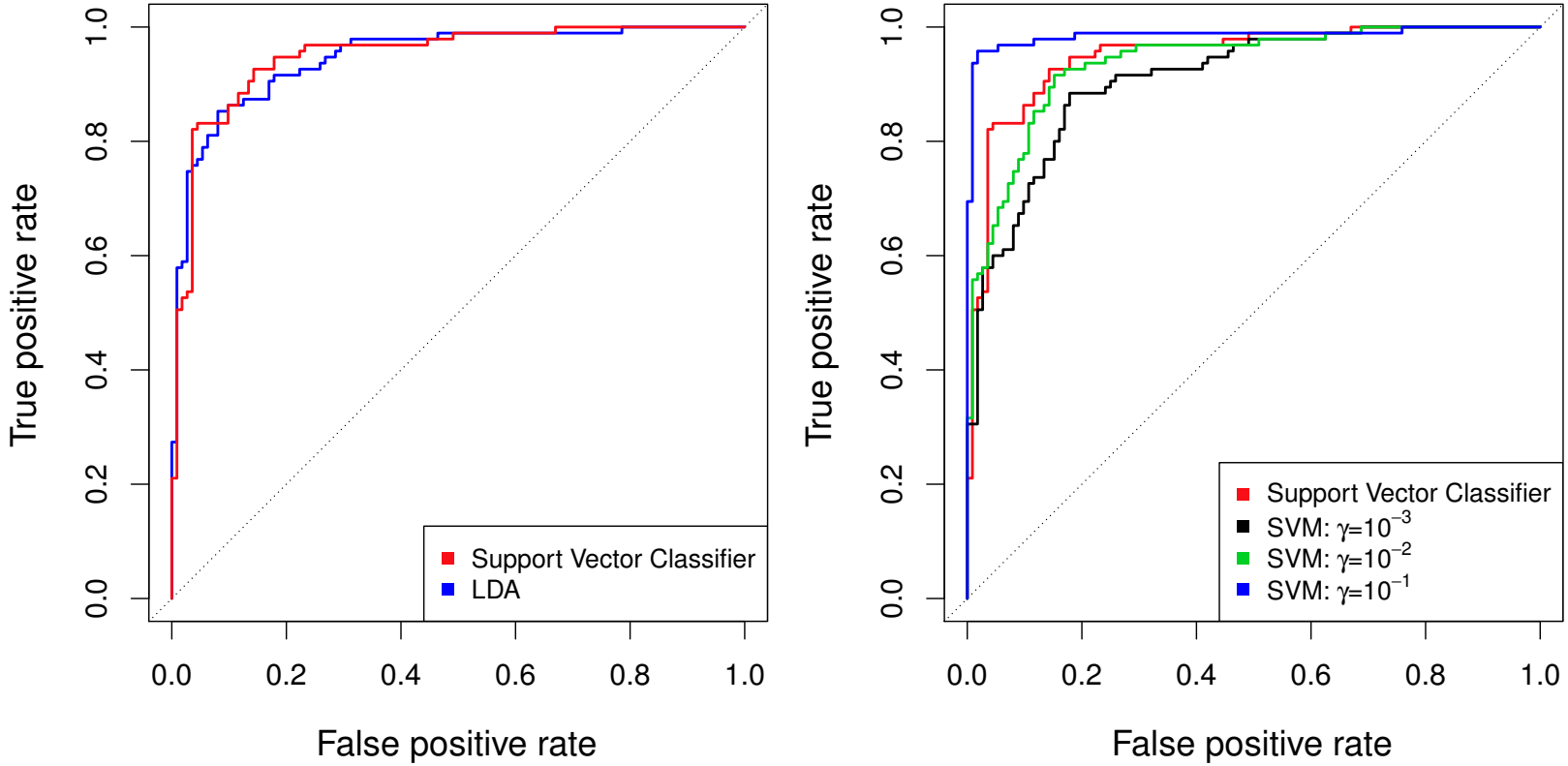
$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \hat{\alpha}_i K(x, x_i),$$

implicit feature space; very high dimensional.

- Controls variance by squaring down most dimenions severely.



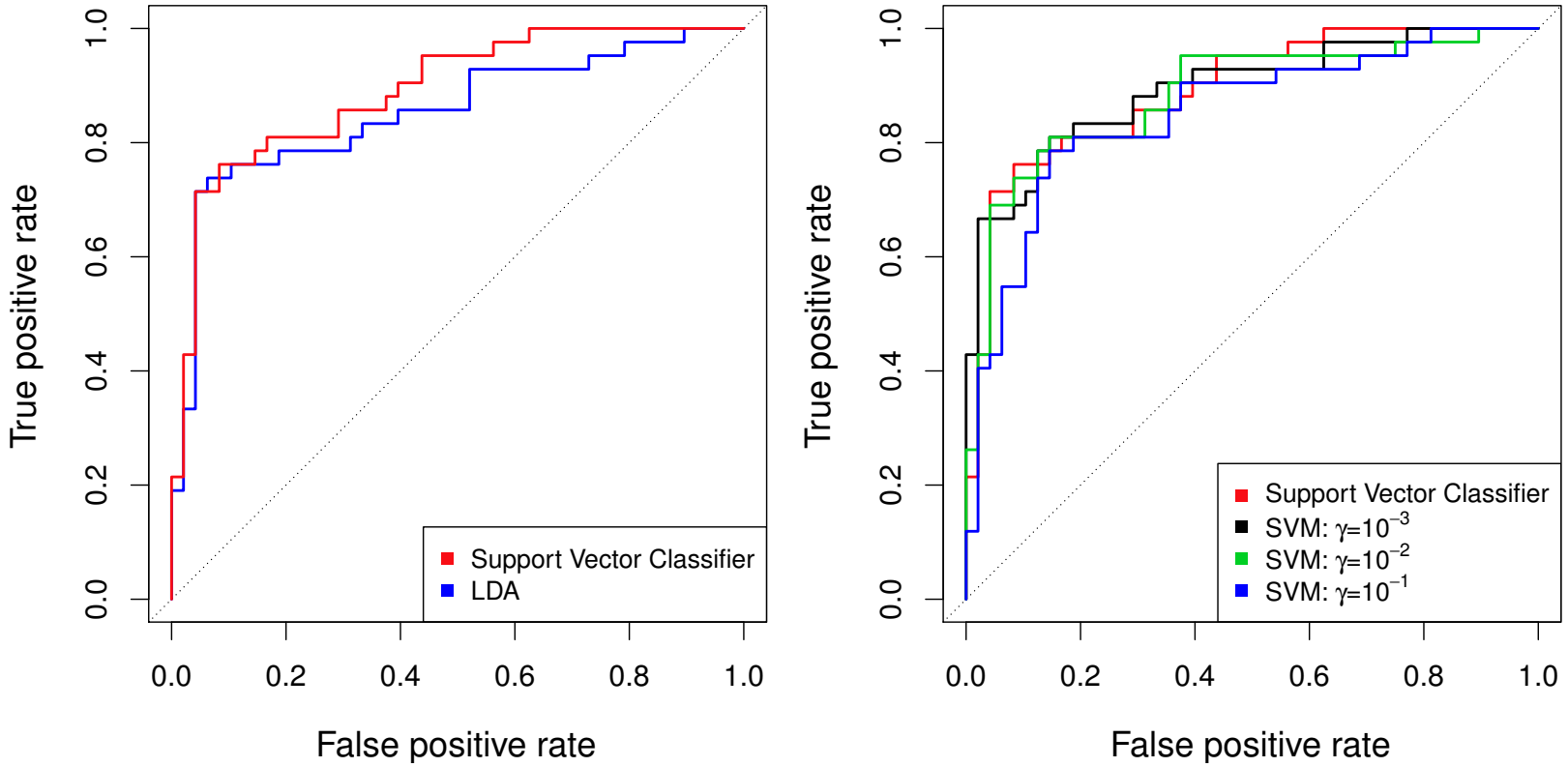
Example - Heart Data



ROC curves on Training data

- ROC curve is obtained by changing the threshold 0 to threshold t in $\hat{f}(X) > t$, and recording false positive and true positive rates as t varies.

Example - Heart Data



ROC curves on Testing data

- Recall that last time the output classifier of SVM is based on a function (linear or nonlinear) $f(x)$ which has real values. The classifier we defined is $\text{sign}\,(f(x))$. Equivalently speaking,

$$f(x) \geq 0 \Rightarrow x \in \text{ class 1, or "Positive"},$$
$$f(x) < 0 \Rightarrow x \in \text{ class 2, or "Negative"}.$$

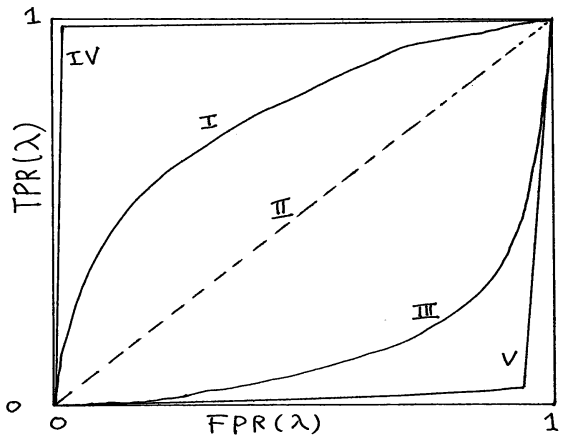
- Sometimes, we hope to give one class some preference. For example, when we design a fire alarm, it won't hurt too much if the alarm bell rings when there is indeed no fire; but not vise versa!!
- Therefore we set a threshold λ, and the above design of classifier is only a special case of $\lambda = 0$.

$$f(x) \geq \lambda \Rightarrow x \in \text{ class 1, or "Positive"},$$
$$f(x) < \lambda \Rightarrow x \in \text{ class 2, or "Negative"}.$$

For a specified λ, let $f_{\mathcal{C}}(x) = \text{P}$ when $f(x) \geq \lambda$, and $f_{\mathcal{C}}(x) = \text{N}$ when $f(x) < \lambda$.

- We see that for $\lambda = -\infty$, the classifier predicts everything as positive. By the previous definition, TPR $= TPR(\lambda) = 1$, and $FPR = FPR(\lambda) = 1$. Of course, this is just a useless predictor.
- We see that for $\lambda = \infty$, the classifier predicts everything as negative. Then $TPR = TPR(\lambda) = 0$, and $FPR = FPR(\lambda) = 0$. Of course, this is another useless predictor.
- We see that as λ changes from ∞ to $-\infty$, $FPR(\lambda)$ and $TPR(\lambda)$ both increase.

The Receiver Operating Characteristics (ROC) Curve



ROC curves on Training data

- "The Receiver Operating Characteristics (ROC) curves were originally developed in signal detection theory in connection with radio signals, and have been used since then in many other applications, in particular for medical decision-making. Over the last few years, they have found increased interest in the machine learning and data mining communities for model evaluation and selection." Cited from C. Cortes and M. Mohri, NIPS 2004

UNIVERSITY OF ALBERTA
EDMONTON·ALBERTA·CANADA

The Receiver Operating Characteristics (ROC) Curve

1. The "Perfect" Classifiers (IV and V)

Curves **IV** and **V** are the gold standard.

- They have a very high **True Positive Rate (TPR)** while maintaining a near-zero **False Positive Rate (FPR)**.
- Visually, they form a "sharp elbow." This means the model can catch almost 100% of the positive cases (like sick patients) without accidentally flagging healthy people as sick.
- These models have an **AUC (Area Under the Curve)** very close to **1.0**.

2. The "Typical" Classifier (I)

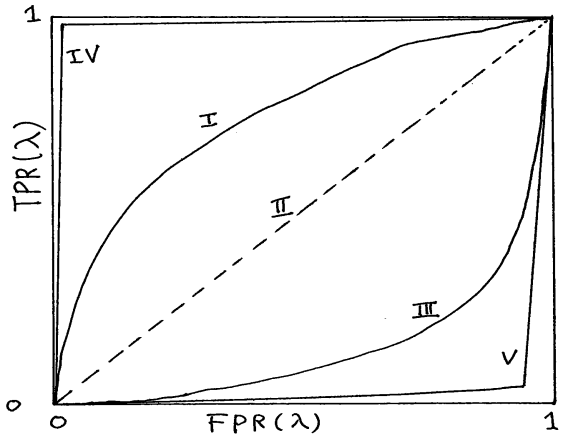
Curve **I** represents a realistic, functional model.

- It is better than guessing, but it faces a clear trade-off: to catch more positives (moving up the Y-axis), it inevitably has to accept more false alarms (moving right on the X-axis).
- This is usually the type of curve you'll see in real-world machine learning projects.

3. The "Useless" Classifier (II)

Curve **II** is the diagonal dashed line.

- This represents a model that is **randomly guessing**.
- For every 10% increase in catching real positives, you get a 10% increase in false alarms. It has an **AUC of 0.5**.
- If your model looks like this, it hasn't learned any useful patterns from your data.



ROC curves on Training data

4. The "Inverse" Classifier (III)

Curve **III** is interesting because it's actually "worse" than guessing (it bows toward the bottom-right).

- This model is consistently contradicting the truth.
- **The "Secret" Benefit:** As the slide notes, this is actually a great model if used **reversely**. If the model says "Positive," you just record it as "Negative," and it suddenly transforms into Curve IV!

- Here we have five ROC curves. Curve I is typical. Curve II indicates a total random hence useless classifier, since for any specific , it always predict the actually positive observations as P with probability 0.5, and N with probability 0.5; same for the actually negative observations. Curve III is kind of useful in the sense that one benefits by using it reversely. IV is nearly a perfect classifier, so is V.

SVMs: More than 2 classes

1. One-versus-All (OVA) / One-versus-Rest

This is the "Me against the world" approach.

- **How it works:** If you have 3 classes (Red, Green, Blue), you build 3 separate SVMs:
  1. Red vs. (Green + Blue)
  2. Green vs. (Red + Blue)
  3. Blue vs. (Red + Green)
- **The Decision:** When a new point x^* arrives, you run it through all 3 models. You assign it to the class whose model gave the **highest confidence score** ($f_k(x^*)$ is largest).

2. One-versus-One (OVO)

This is the "Tournament" approach.

- **How it works:** You build a separate SVM for **every possible pair** of classes.
- **The Math:** For K classes, you build $\binom{K}{2}$ classifiers. For 3 classes (R, G, B), you'd have 3 pairs: (R vs G), (R vs B), and (G vs B). For 10 classes, you'd need **45** classifiers!
- **The Decision:** Each classifier "votes." You assign the new point to the class that **wins the most pairwise competitions**.

- The SVM as defined works for $K = 2$ classes. What do we do if we have $K > 2$ classes?
- OVA - One versus All. Fit K different 2-class SVM classifiers $\hat{f}_k(x),\ k = 1, \cdots, K$; each class versus the rest. Classify x^* to the class for which $\hat{f}_k(x^*)$ is largest.
- OVO - One versus One. Fit all $\binom{K}{2}$ pairwise classifiers $\hat{f}_{kl}(x)$. Classify x^* to the class that wins the most pairwise competitions.
- Which one to choose? If K is not too large, use OVO.

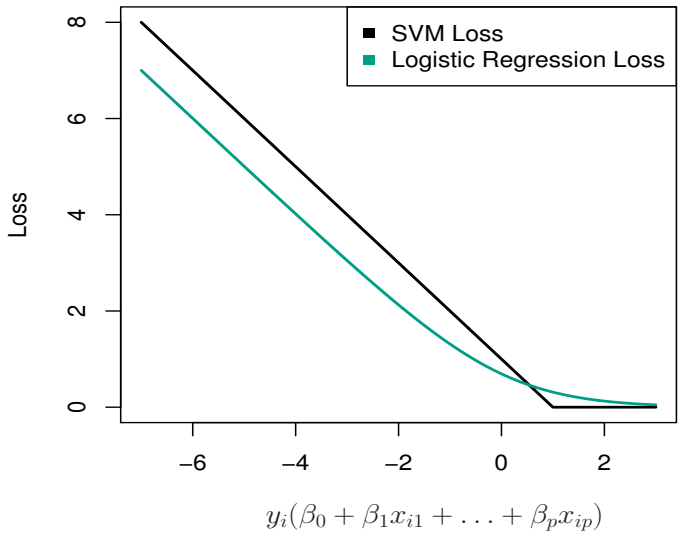
Support Vector Machine Versus Logistic Regression

- Let $f(X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$, support vector machine can be rephrased as

$$\text{minimize}_{\beta_0, \beta_1, \cdots, \beta_p} \left\{ \sum_i \max[0, 1 - y_i f(x_i)] + \lambda \sum_j \beta_j^2 \right\},$$

where γ is tuning parameter.

- This has the form of loss plus penalty.
- The loss is known as hinge loss.
- Very similar to the loss in logistic regression (negative log-likelihood).



UNIVERSITY OF ALBERTA
EDMONTON·ALBERTA·CANADA

Kernels and Support Vector Machine

- When classes are (nearly) separable, SVM does better than LR. So does LDA.
- When not, LR (with ridge penalty) and SVM very similar.
- If you wish to estimate probabilities, LR is the choice.
- For nonlinear boundaries, kernel SVMs are popular. Can use kernels with LR and LDA as well, but computations are more expensive.

UNIVERSITY OF ALBERTA
EDMONTON·ALBERTA·CANADA

Summary and Remark

- Feature expansion
- Kernel trick
- More than 2 classes
- Read textbook Chapter 9 and `R` code